

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikrokontroler

Kelvin Krismora Gultom - 5024231062

2026

1 Langkah-Langkah Percobaan

Percobaan 1:

- Siapkan board STM32 Blackpill, programmer ST-Link V2, kabel jumper, dan kabel USB.
- Sambungkan pin SWDIO, SWCLK, GND, dan 3.3V pada ST-Link ke Blackpill dengan kabel jumper, kemudian colokkan ST-Link ke laptop.
- Atur `platformio.ini` agar memakai board `blackpill_f401cc`, framework Arduino, dan protokol upload `stlink`.
- Tulis program blink untuk LED bawaan board (PC13) pada file `src/main.cpp`, lalu lakukan build dan upload sampai LED berkedip setiap 0,5 detik.

Percobaan 2:

- Siapkan board ESP32 DevKit dan hubungkan langsung ke laptop lewat kabel USB sampai portnya terdeteksi.
- Ubah `platformio.ini` menjadi board `esp32dev` dengan framework `espidf`, dan ganti file utama dari `main.cpp` menjadi `src/main.c`.
- Ketik program blink untuk LED onboard (GPIO 2) di `src/main.c`, kemudian jalankan build dan upload sampai LED berkedip.

Percobaan 3:

- Siapkan ESP32 DevKit, breadboard, dua pushbutton, dua resistor 10k, dan kabel jumper male-male.
- Rangkai konfigurasi eksternal: tombol pertama dihubungkan ke GPIO 32 (ke 3.3V melalui resistor 10k, kaki satunya ke GND), sedangkan tombol kedua dihubungkan ke GPIO 33 (ke GND melalui resistor 10k, kaki satunya ke 3.3V).
- Kembalikan `platformio.ini` ke framework Arduino dan buat ulang file `src/main.cpp`.
- Masukkan kode pembacaan eksternal ke `src/main.cpp`, upload, lalu periksa hasil pembacaan tombol pada Serial Monitor.
- Ubah rangkaian ke konfigurasi internal dengan melepas kedua resistor 10k dari breadboard, tanpa menggeser kabel tombol yang lain.
- Ganti isi `src/main.cpp` dengan program pull-up dan pull-down internal, lalu flash ulang dan bandingkan hasilnya di Serial Monitor.

Percobaan 4:

- Siapkan ESP32 DevKit, breadboard, satu pushbutton, dan kabel jumper.
- Pasang satu kaki pushbutton ke GPIO 25 dan kaki pasangannya langsung ke jalur GND, sehingga fitur pull-up internal dapat dipakai.

- Masukkan kode program penghitung debounce ke `src/main.cpp` dengan pengaturan `platformio.ini` yang sama seperti percobaan sebelumnya.
- Upload program, lalu uji dengan menekan tombol secara cepat dan secara pelan. Variasikan nilai `DEBOUNCE_DELAY` (50 ms, 10 ms, dan 200 ms), flash ulang setiap kali diubah, dan amati perbandingan angkanya di Serial Monitor.

Percobaan 5:

- Siapkan ESP32 DevKit, breadboard, satu sensor cahaya LDR, satu resistor 10k, dan kabel jumper.
- Rangkai pembagi tegangan: satu kaki LDR ke 3.3V, sedangkan kaki lainnya ke GPIO 34 dan sekaligus ke GND melalui resistor 10k.
- Masukkan kode pembacaan ADC ke `src/main.cpp` dengan `platformio.ini` tetap memakai framework Arduino.
- Build dan upload program ke board, lalu pantau perubahan nilai analog dan tegangan di Serial Monitor ketika LDR ditutup tangan maupun disinari cahaya.

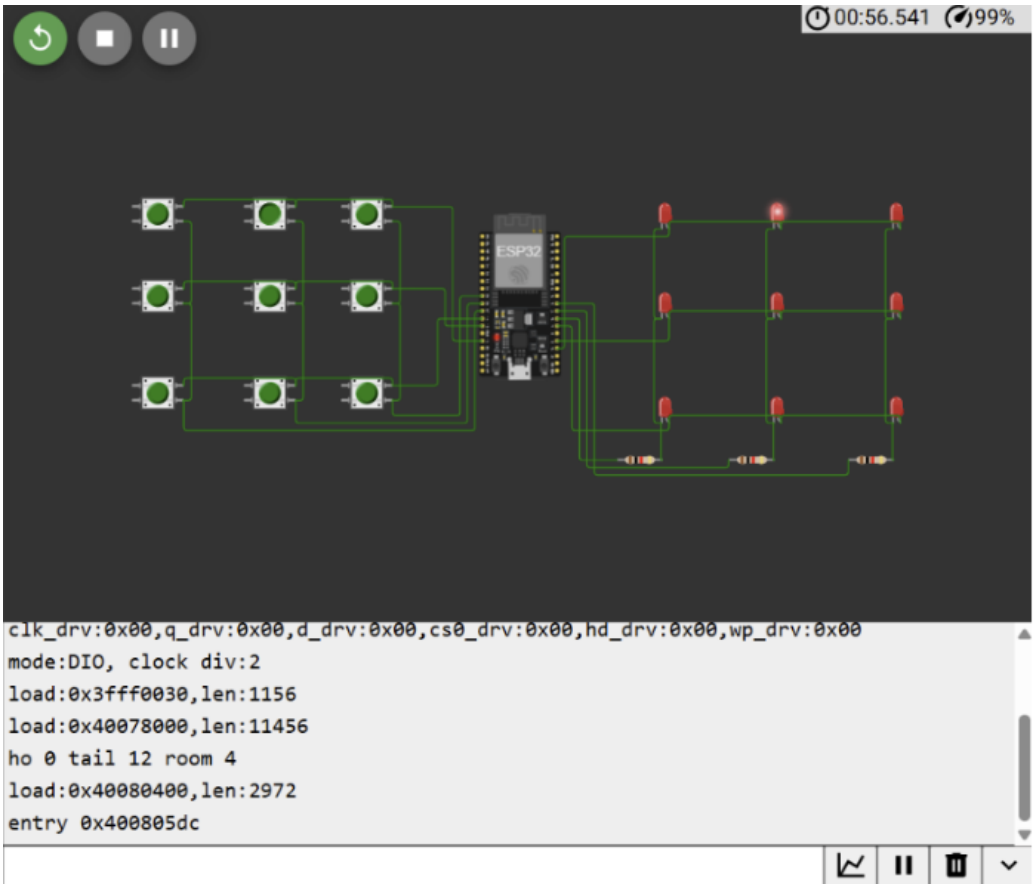
2 Analisis Hasil Percobaan

Pada percobaan pertama, program blink di-flash ke board STM32 Blackpill melalui ST-Link dengan framework Arduino. LED bawaan di pin PC13 diharapkan menyala dan mati secara bergantian setiap 500 ms, dan hasil praktikum sesuai dengan harapan tersebut. Hal ini menandakan bahwa koneksi jalur SWD dari ST-Link sudah terpasang dengan benar. Percobaan kedua bertujuan menjalankan program blink pada ESP32 dengan framework native ESP-IDF. LED onboard di GPIO 2 diharapkan berkedip setiap 0,5 detik, dan hasilnya pun sesuai. Dari percobaan ini terlihat bahwa ESP32 dapat langsung di-flash lewat kabel USB tanpa perangkat tambahan, dan struktur program berbasis FreeRTOS terbukti mampu mengendalikan GPIO dasar sebaik `setup()` dan `loop()` pada Arduino. Pada percobaan ketiga, kerja resistor pull-up (GPIO 32) dan pull-down (GPIO 33) pada tombol dibandingkan, baik yang dirangkai secara eksternal maupun yang diaktifkan secara internal lewat program. Kedua konfigurasi diharapkan menampilkan pembacaan yang persis sama di Serial Monitor, dan hasil pengujian memang demikian. Ini menunjukkan bahwa resistor internal bawaan ESP32 berfungsi sama seperti resistor fisik 10k, yaitu menjaga nilai pin agar tidak *floating* (acak): pull-up terbaca LOW saat tombol ditekan, sedangkan pull-down terbaca HIGH saat ditekan. Percobaan keempat berfokus pada efek getaran kontak mekanik (*bouncing*) ketika tombol ditekan serta cara mengatasinya dengan teknik *debouncing* berbasis software pada GPIO 25. Penghitung dengan debounce diharapkan hanya bertambah satu setiap tombol ditekan, sedangkan penghitung tanpa debounce bisa melonjak lebih dari satu, dan hasil percobaan sesuai dengan harapan itu. Selain itu, variasi nilai delay (10 ms, 50 ms, dan 200 ms) menunjukkan bahwa waktu tunggu yang terlalu singkat (10 ms) masih mudah lolos oleh getaran, sedangkan yang terlalu lama (200 ms) membuat tombol kurang responsif ketika ditekan dengan cepat, sehingga 50 ms menjadi nilai yang paling pas. Pada percobaan kelima, input analog (ADC) di GPIO 34 dibaca melalui rangkaian pembagi tegangan yang terdiri dari sensor cahaya LDR dan resistor 10k. Nilai ADC (rentang 0–4095) dan tegangan di Serial Monitor diharapkan berubah mengikuti terang-gelapnya cahaya saat LDR ditutup tangan atau disinari lampu, tetapi hasil

praktikum tidak sesuai karena nilainya stagnan dan tidak berubah. Padahal rangkaian sudah dicek dan dinyatakan benar, sehingga penyebab yang mungkin adalah kaki LDR atau kabel jumper yang tidak menancap sempurna pada lubang breadboard yang longgar sehingga tegangan tidak sampai ke pin, atau komponen LDR yang dipakai memang sudah rusak sejak awal sehingga nilai resistansinya tidak berubah saat terkena cahaya.

3 Hasil Tugas Modul

3.1 Simulasi Rangkaian



Gambar 1: Hasil Rangkaian

3.2 Source code yang digunakan

```
// Pin matrix pushbutton
const uint8_t barisTombol[3] = {13, 12, 14};
const uint8_t kolomTombol[3] = {27, 26, 25};

// Pin matrix LED
const uint8_t barisLed[3] = {15, 2, 4};
const uint8_t kolomLed[3] = {16, 17, 5};

const unsigned long JEDA_DEBOUNCE = 10; // waktu tunggu sinyal stabil (ms)

bool tombolAktif[3][3]           = {};  // status tombol setelah debounce
bool bacaanSebelumnya[3][3]      = {};  // bacaan mentah pada siklus lalu
```

```
unsigned long waktuBerubah[3][3] = {};  // waktu terakhir bacaan berubah

void setup() {
  for (uint8_t i = 0; i < 3; i++) {
    // Sisi tombol: baris sebagai output, kolom sebagai input pull-up
    pinMode(barisTombol[i], OUTPUT);
    digitalWrite(barisTombol[i], HIGH);
    pinMode(kolomTombol[i], INPUT_PULLUP);

    // Sisi LED: semua baris dan kolom dalam kondisi mati
    pinMode(barisLed[i], OUTPUT);
    digitalWrite(barisLed[i], LOW);
    pinMode(kolomLed[i], OUTPUT);
    digitalWrite(kolomLed[i], HIGH);
  }
}

void bacaMatriksTombol() {
  for (uint8_t b = 0; b < 3; b++) {
    digitalWrite(barisTombol[b], LOW);  // aktifkan satu baris
    delayMicroseconds(10);

    for (uint8_t k = 0; k < 3; k++) {
      bool tertekan = !digitalRead(kolomTombol[k]);  // LOW berarti ditekan

      // Catat waktu setiap kali bacaan berubah
      if (tertekan != bacaanSebelumnya[b][k]) {
        waktuBerubah[b][k] = millis();
      }

      // Terima perubahan hanya jika sinyal sudah stabil cukup lama
      bool sudahStabil = (millis() - waktuBerubah[b][k]) > JEDA_DEBOUNCE;
      if (sudahStabil) {
        tombolAktif[b][k] = tertekan;
      }

      bacaanSebelumnya[b][k] = tertekan;
    }

    digitalWrite(barisTombol[b], HIGH);  // nonaktifkan baris kembali
  }
}

void matikanSemuaBarisLed() {
  for (uint8_t i = 0; i < 3; i++) {
    digitalWrite(barisLed[i], LOW);
  }
}

void perbaruiMatriksLed() {
  for (uint8_t b = 0; b < 3; b++) {
    matikanSemuaBarisLed();

    // Kolom LOW berarti LED pada kolom tersebut menyala
```

```
67      for (uint8_t k = 0; k < 3; k++) {
68        digitalWrite(kolomLed[k], tombolAktif[b][k] ? LOW : HIGH);
69      }
70
71      digitalWrite(barisLed[b], HIGH);  // nyalakan baris yang sedang dipindai
72      delay(2);
73    }
74  }
75
76  void loop() {
77    bacaMatriksTombol();
78    perbaruiMatriksLed();
79  }
```

3.3 Pertanyaan Analisis

- **Mengapa debouncing tetap dibutuhkan pada rangkaian matrix pushbutton ini, meskipun pada satu waktu umumnya hanya satu tombol yang ditekan? Kaitkan dengan penyebab bouncing pada C.5 Debouncing.**
  Ketika tombol dipencet, pelat logam di dalamnya ikut bergetar (*bouncing*) sehingga menghasilkan sinyal putus-sambung yang sangat cepat. Tanpa debouncing, satu kali pencet bisa dibaca berkali-kali oleh mikrokontroler, akibatnya LED menjadi kacau atau berkedip sendiri.

- **Jelaskan konfigurasi pull-up atau pull-down (sesuai rancangan Anda) pada pembacaan kolom tombol, bagaimana kondisi pin (HIGH/LOW) saat tombol tidak ditekan, dan mengapa pin tidak boleh dibiarkan *floating*.**
  Pada konfigurasi pull-up, pin kolom berada pada kondisi HIGH selama tombol tidak dipencet (atau LOW bila memakai pull-down). Pin tidak boleh dibiarkan mengambang (*floating*) karena mudah menangkap gangguan listrik (*noise*), sehingga status tombol terbaca acak dan LED dapat menyala sendiri walaupun tidak ada tombol yang dipencet.

- **Jika salah satu tombol pada matrix diganti dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), jelaskan perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), serta mengapa ADC mampu merepresentasikan lebih dari dua kondisi.**
  Digital input hanya mengenali dua keadaan tegas, yaitu HIGH atau LOW (nyala atau mati), sedangkan analog input melalui ADC membaca perubahan tegangan secara bertingkat dan kontinu. Pada ESP32 dengan resolusi 12-bit, tegangan 0–3.3V dikonversi menjadi angka dalam rentang 0–4095, sehingga berbagai tingkat intensitas cahaya pada LDR dapat dibedakan.

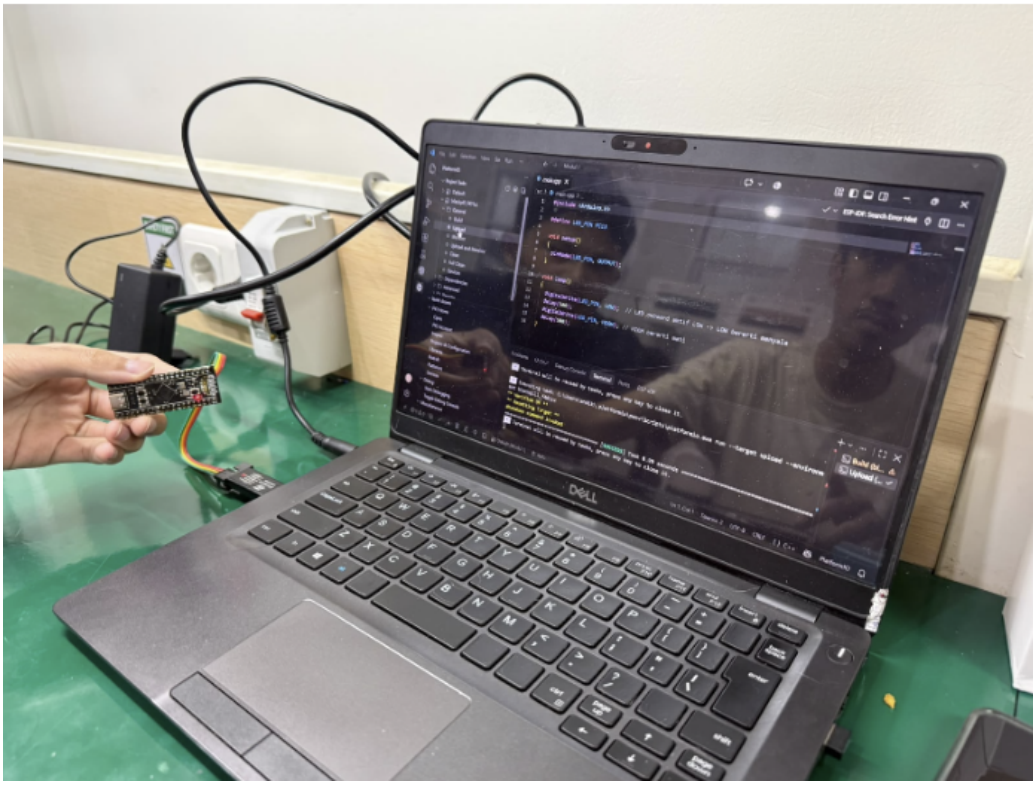
4 Kesimpulan

Praktikum Modul 1 berhasil memperkenalkan dasar pemrograman mikrokontroler menggunakan PlatformIO. Program blink dapat dijalankan pada STM32 Blackpill melalui ST-Link dengan framework Arduino, serta pada ESP32 dengan framework ESP-IDF yang berbasis FreeRTOS. Pengujian pull-up dan pull-down menunjukkan bahwa resistor internal ESP32 bekerja sama seperti resistor eksternal

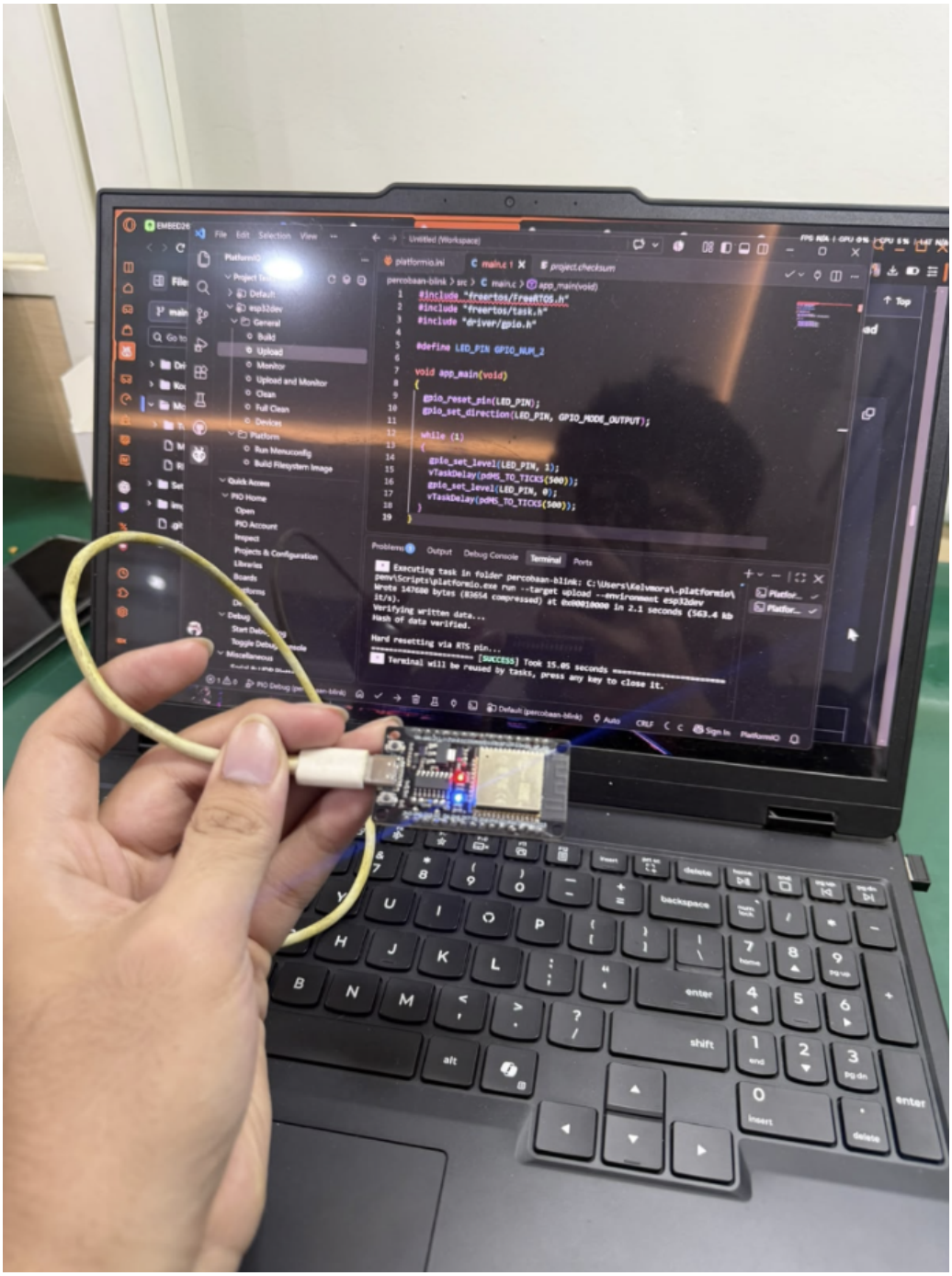
10k, yaitu menjaga pin agar tidak *floating*. Percobaan debouncing membuktikan bahwa getaran kontak mekanik dapat menyebabkan satu tekanan terbaca berkali-kali, dan nilai delay 50 ms merupakan keseimbangan terbaik antara kestabilan pembacaan dan responsivitas tombol. Pada percobaan ADC, nilai pembacaan LDR tidak berubah sehingga hasilnya belum sesuai harapan. Kemungkinan penyebabnya adalah sambungan yang longgar pada breadboard atau LDR yang rusak. Tugas modul berupa matrix pushbutton 3×3 yang mengendalikan matrix LED 3×3 berhasil disimulasikan dengan penerapan debouncing pada setiap tombol.

5 Lampiran

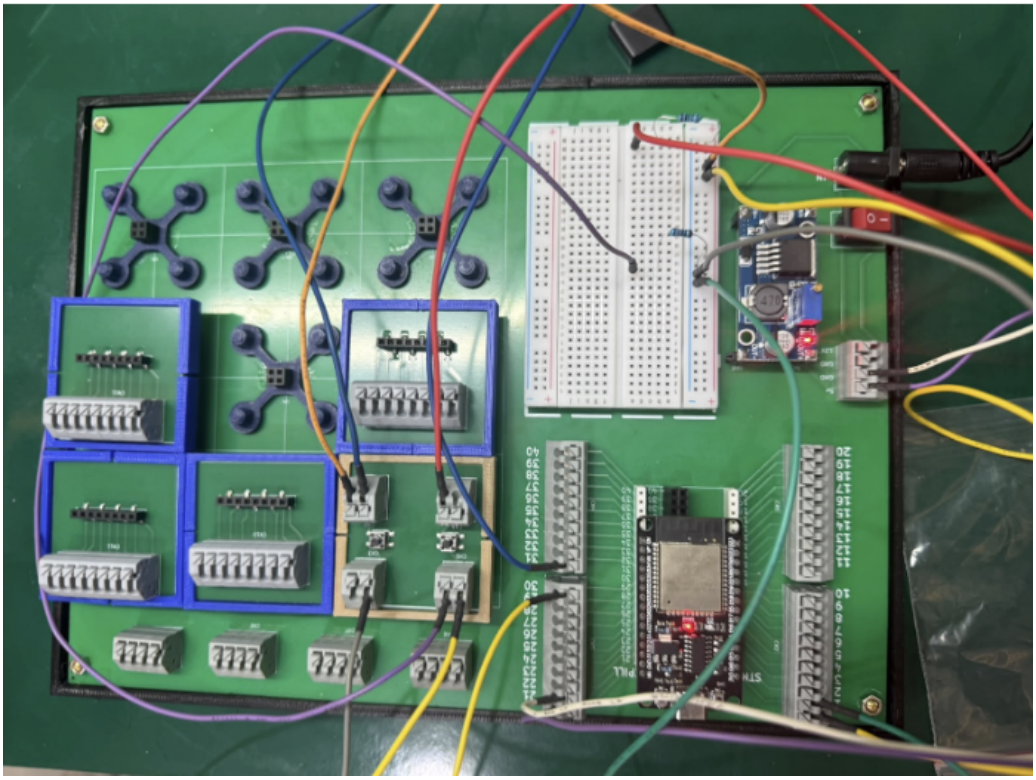
5.1 Dokumentasi saat praktikum



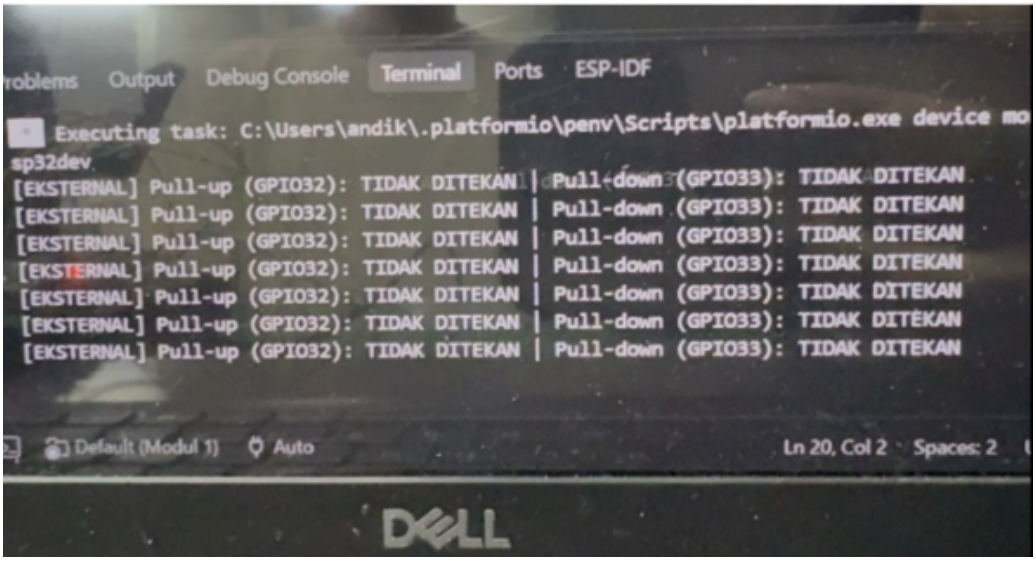
Gambar 2: Percobaan 1



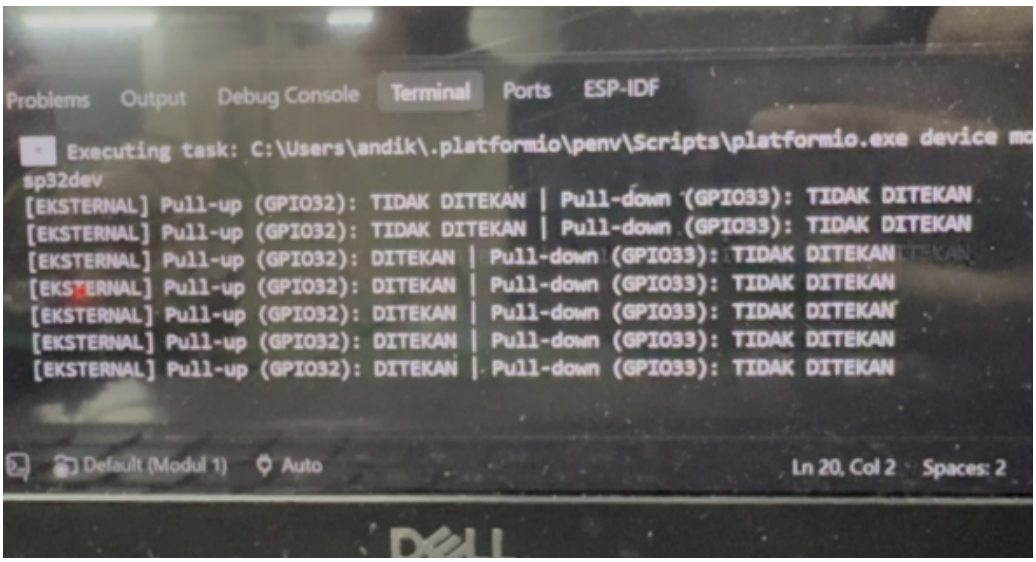
Gambar 3: Percobaan 2



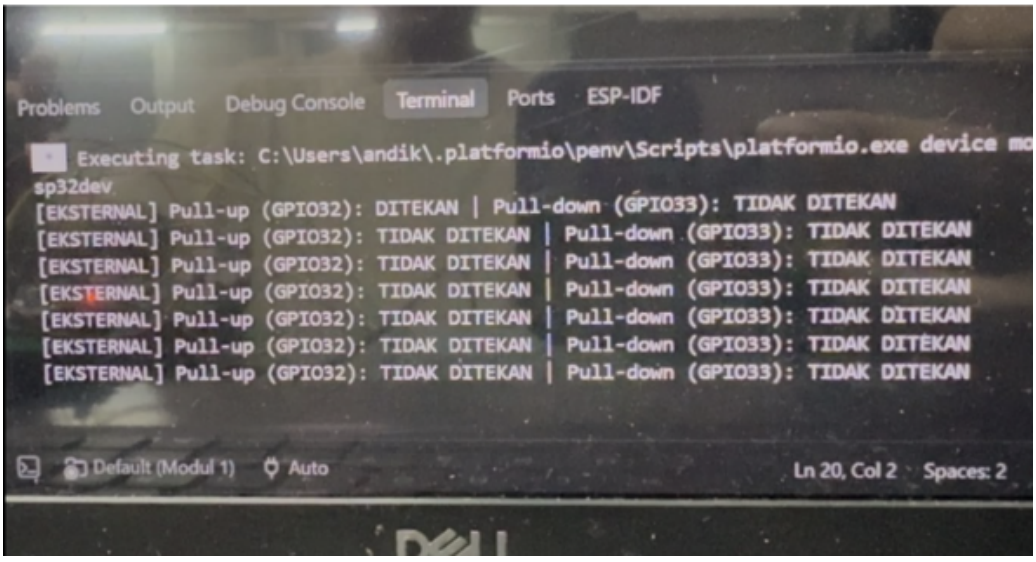
Gambar 4: Percobaan 3(a)



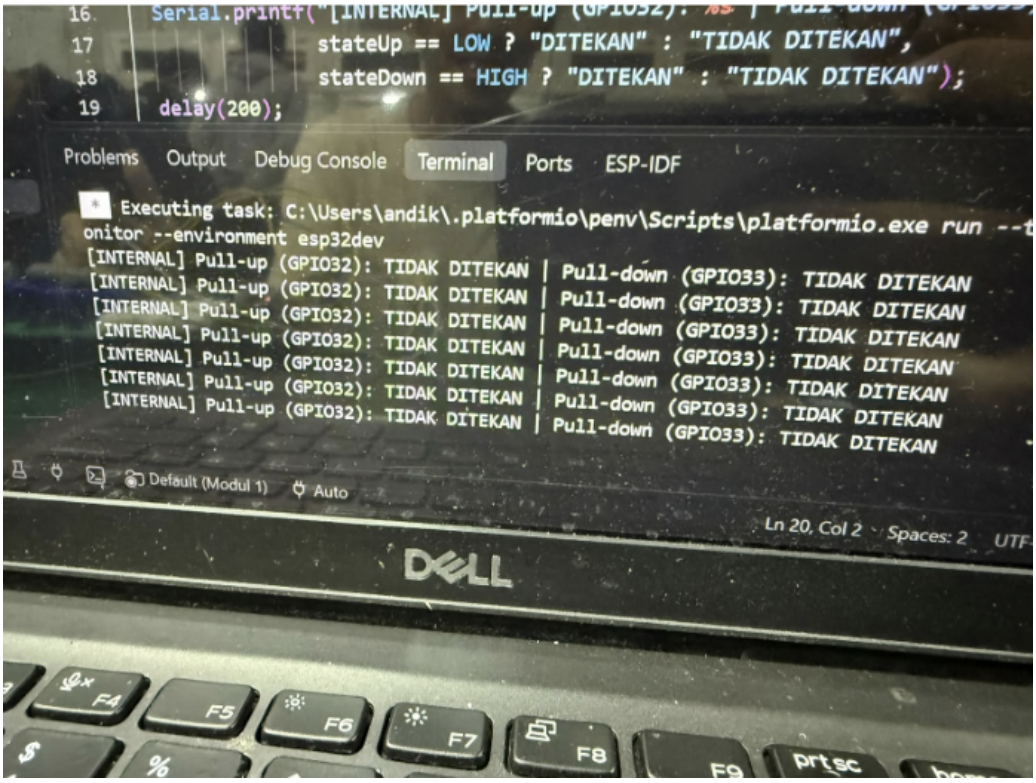
Gambar 5: Percobaan 3(a) - State 0



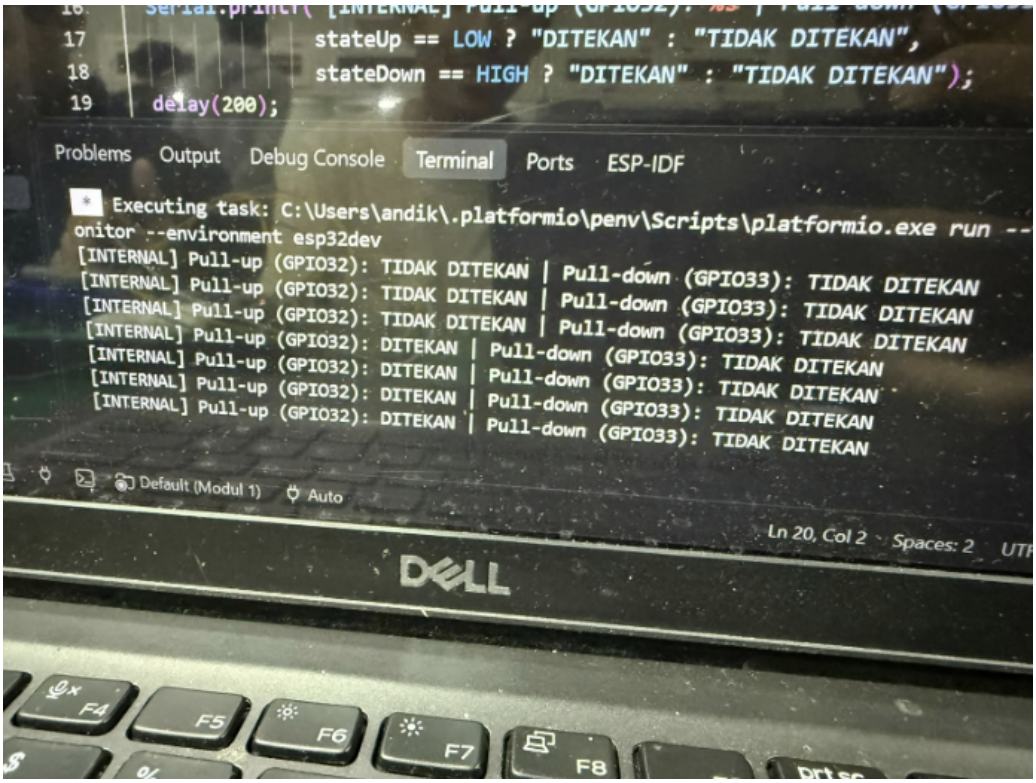
Gambar 6: Percobaan 3(a) - State 1



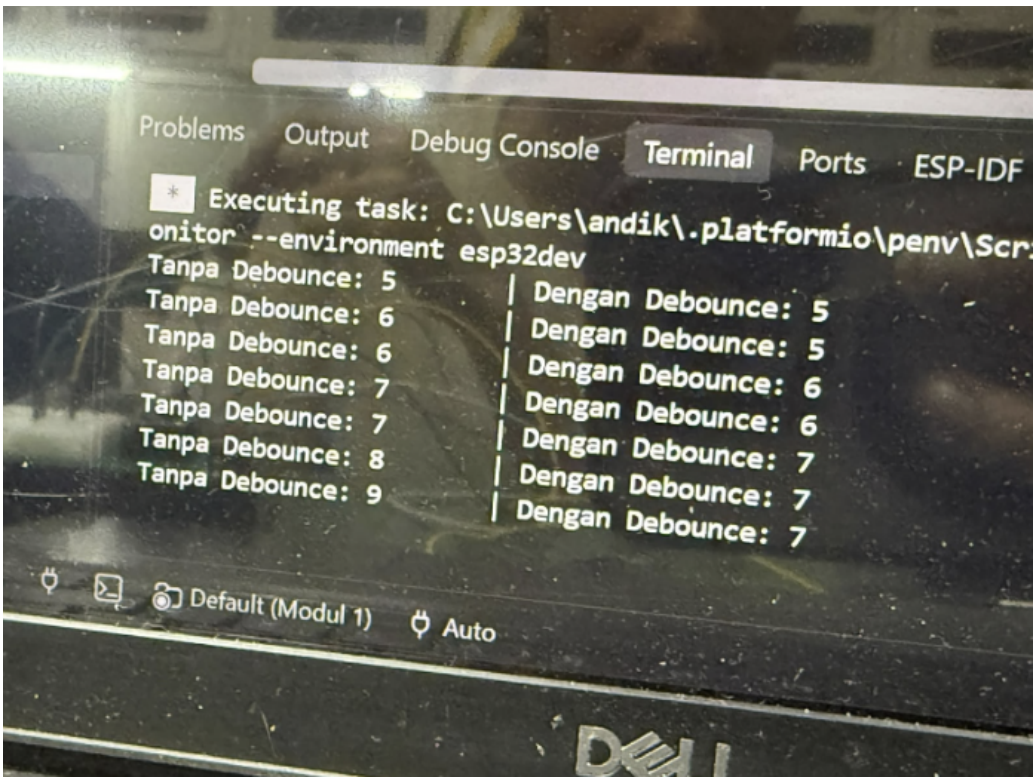
Gambar 7: Percobaan 3(a) - State 2



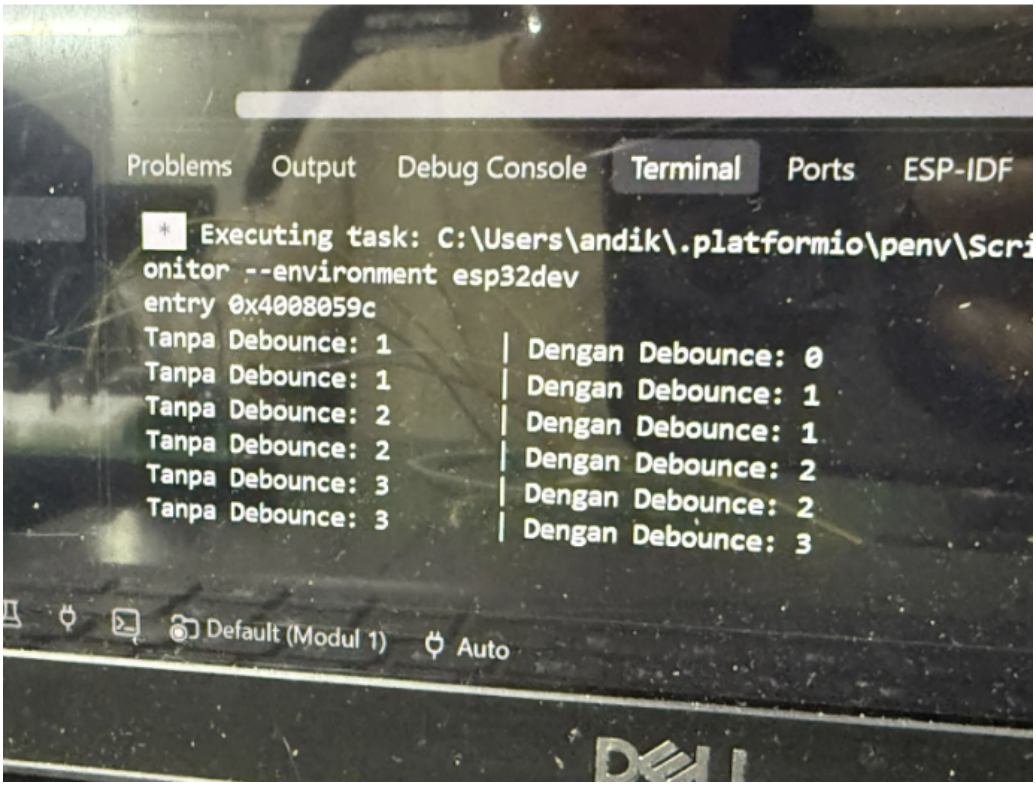
Gambar 8: Percobaan 3(b) - State 0



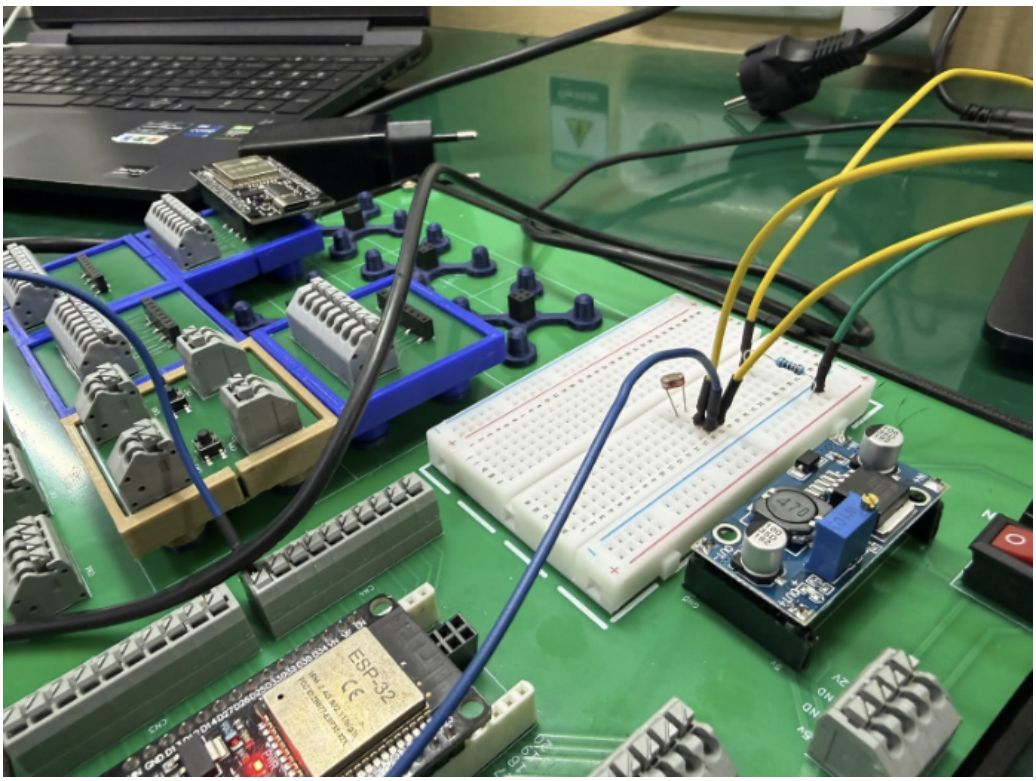
Gambar 9: Percobaan 3(b) - State 1



Gambar 10: Percobaan 4 - Ditekan Cepat



Gambar 11: Percobaan 4 - Ditekan Lambat



Gambar 12: Percobaan 5